

Centro Universitário UniFECAF

Curso de Engenharia da Computação

SISTEMA AUTOMOTIVO – GESTÃO DE ESTOQUE DE VEÍCULOS AUTOSTOCK

RELATÓRIO EXPLICATIVO

Adriano Mantoan

RA:175928

São Paulo

Agosto de 2026

Sumário

1. Introdução.....	1
2. Objetivo	1
3. Levantamento de Requisitos	2
3.1 Requisitos Funcionais	3
3.2 Requisitos Não Funcionais	4
3.3 Principais Entidades do Sistema.....	4
4. Ferramentas Utilizadas	4
5. Desenvolvimento das Atividades	5
5.1 Organização inicial do projeto	5
5.2 Configuração do banco de dados	6
5.3 Desenvolvimento da interface e execução do sistema.....	7
5.4 Cadastro de marcas e modelos	8
5.5 Cadastro de veículos	9
5.6 Consulta e filtros do estoque.....	11
5.7 Edição das informações.....	14
5.8 Exclusão de veículos	15
6. Aplicação dos Conceitos de Programação Orientada a Objetos.....	16
6.1 Classes e objetos.....	16
6.2 Atributos e métodos	17
6.3 Encapsulamento	17
6.4 Herança	18
6.5 Relacionamento entre classes	19
7. Organização das Camadas da Aplicação	20
8. Testes e Validação	21
9. Dificuldades Encontradas	22
10. Análise dos Resultados	23
11. Conclusão	24
12. Referências.....	25

1. Introdução

A tecnologia está cada vez mais presente na organização das empresas e pode ajudar no controle de informações que antes eram registradas de forma manual ou em diferentes locais. Durante a disciplina de Programação Orientada a Objetos, tive a oportunidade de compreender melhor como os conceitos estudados podem ser utilizados na criação de uma aplicação.

Para colocar esses conhecimentos em prática, desenvolvi o AutoStock, um sistema automotivo voltado para a gestão de estoque de veículos. A proposta foi desenvolver uma aplicação capaz de cadastrar marcas, modelos e veículos, além de permitir a consulta, edição, exclusão e utilização de filtros para facilitar a localização das informações.

Durante o desenvolvimento, procurei aplicar os conhecimentos adquiridos na disciplina e também compreender melhor como diferentes tecnologias trabalham em conjunto. Para isso, utilizei Java com Spring Boot no desenvolvimento da aplicação, MySQL para armazenamento das informações e uma interface web para interação com o usuário.

O desenvolvimento também foi importante para entender, na prática, que uma aplicação não depende apenas de uma única classe ou arquivo. Foi necessário fazer com que diferentes partes do projeto se comunicassem corretamente para que uma informação cadastrada na tela pudesse ser processada pela aplicação e posteriormente armazenada no banco de dados.

2. Objetivo

O objetivo deste trabalho foi desenvolver um sistema para gerenciamento de estoque de veículos utilizando conceitos de Programação Orientada a Objetos.

O sistema desenvolvido recebeu o nome de AutoStock e foi planejado para permitir o cadastro e gerenciamento das principais informações relacionadas aos veículos de uma empresa do setor automotivo.

Entre as funcionalidades desenvolvidas estão o cadastro de marcas e modelos, cadastro de veículos, visualização do estoque, alteração das informações cadastradas, exclusão de registros e realização de pesquisas utilizando diferentes filtros.

Além do funcionamento do sistema, o projeto teve como objetivo permitir que eu pudesse aplicar e compreender melhor conceitos estudados durante a disciplina, principalmente classes, objetos, atributos, métodos, encapsulamento e relacionamento entre classes.

3. Levantamento de Requisitos

Antes de iniciar o desenvolvimento, foi necessário compreender quais informações e funcionalidades seriam necessárias para um sistema de controle de estoque automotivo.

Para realizar esse levantamento, considerei uma situação simulada na qual um responsável pelo estoque de veículos informaria suas principais necessidades.

Foram elaboradas as seguintes perguntas e respostas:

1. Quais informações precisam ser cadastradas para cada veículo?

Resposta: Marca, modelo, ano, cor, preço, quilometragem, status e uma observação quando necessário.

2. O sistema precisa permitir o cadastro de diferentes marcas?

Resposta: Sim. As marcas devem poder ser cadastradas para posteriormente serem utilizadas no cadastro dos modelos.

3. Os modelos devem estar relacionados a uma marca?

Resposta: Sim. Cada modelo deve pertencer a uma marca cadastrada no sistema.

4. É necessário alterar as informações de um veículo depois do cadastro?

Resposta: Sim. Informações como preço, quilometragem, status e demais dados podem precisar ser atualizadas.

5. O sistema precisa permitir a exclusão de veículos?

Resposta: Sim. Deve ser possível remover um veículo quando necessário.

6. Como os veículos devem ser consultados?

Resposta: Os veículos devem aparecer em uma relação de estoque contendo suas principais informações.

7. É necessário pesquisar veículos utilizando filtros?

Resposta: Sim. A pesquisa deve permitir utilizar informações como marca, modelo, ano, status e faixa de preço.

8. Quais situações podem ser informadas no status do veículo?

Resposta: O veículo pode estar disponível, financiado, em negociação, leilão, sinistrado, vendido ou descontinuado, de acordo com sua situação.

9. As informações devem permanecer salvas depois que o sistema for fechado?

Resposta: Sim. Os dados devem ser armazenados em banco de dados para que possam ser consultados posteriormente.

10. O sistema precisa ter uma interface para facilitar sua utilização?

Resposta: Sim. Uma interface simples facilita o cadastro, a consulta e a alteração das informações.

11. O sistema deve impedir que um modelo fique sem uma marca relacionada?

Resposta: Sim. O modelo deve estar relacionado a uma marca existente.

12. O usuário deve conseguir visualizar preço e quilometragem diretamente na relação de veículos?

Resposta: Sim. Essas informações são importantes para o controle do estoque e devem aparecer na listagem.

3.1 Requisitos Funcionais

A partir do levantamento realizado, foram definidos os seguintes requisitos funcionais:

- RF01 – Permitir o cadastro de marcas.
- RF02 – Permitir o cadastro de modelos associados a uma marca.
- RF03 – Permitir o cadastro de veículos.
- RF04 – Exibir os veículos cadastrados no estoque.
- RF05 – Permitir a edição das informações de um veículo.
- RF06 – Permitir a exclusão de veículos vendidos ou descontinuados quando necessário.
- RF07 – Permitir pesquisa por marca.
- RF08 – Permitir pesquisa por modelo.
- RF09 – Permitir pesquisa por ano.
- RF10 – Permitir pesquisa por status.
- RF11 – Permitir pesquisa por faixa de preço.
- RF12 – Armazenar as informações no banco de dados.

3.2 Requisitos Não Funcionais

Também foram considerados alguns requisitos relacionados à utilização e organização do sistema:

1. RNF01 – O sistema deve possuir uma interface simples e de fácil entendimento.
2. RNF02 – As informações devem ser armazenadas em banco de dados MySQL.
3. RNF03 – A aplicação deve utilizar Java e Spring Boot.
4. RNF04 – O projeto deve manter uma estrutura organizada, separando as responsabilidades das classes.
5. RNF05 – O sistema deve apresentar as informações de forma legível no navegador.
6. RNF06 – A aplicação deve manter os dados armazenados mesmo após seu encerramento.
7. RNF07 – Desempenho: O sistema deve realizar as operações de cadastro, consulta, edição, exclusão e filtragem dos veículos em tempo adequado para utilização do estoque.

3.3 Principais Entidades do Sistema

As principais entidades utilizadas no desenvolvimento do AutoStock são Marca, Modelo e Veiculo. A entidade Marca representa os fabricantes dos veículos cadastrados no sistema. A entidade Modelo representa os modelos pertencentes a cada marca, enquanto a entidade Veiculo armazena as informações dos veículos disponíveis no estoque, como ano, cor, preço, quilometragem, status e observação.

Também foi utilizada a classe abstrata EntidadeBase, responsável por disponibilizar o atributo de identificação (id) que é herdado pelas demais entidades. Essa estrutura ajudou a organizar os dados do sistema e a aplicar conceitos de Programação Orientada a Objetos.

4. Ferramentas Utilizadas

Para desenvolver o AutoStock foram utilizadas diferentes ferramentas e tecnologias.

Java 21: utilizado como linguagem principal do projeto. A escolha do Java também permitiu aplicar os conceitos de Programação Orientada a Objetos estudados durante a disciplina.

Spring Boot: utilizado para auxiliar na criação e organização da aplicação Java. Durante o desenvolvimento percebi que o Spring Boot facilita várias configurações necessárias para que a aplicação possa funcionar e se comunicar com outras partes do sistema.

Spring Data JPA/Hibernate: utilizado na comunicação entre as classes Java e as informações armazenadas no banco de dados.

MySQL: utilizado como banco de dados do AutoStock, responsável por armazenar as marcas, modelos e veículos cadastrados.

HTML, CSS e JavaScript: utilizados na construção da interface apresentada no navegador e na interação do usuário com o sistema.

Visual Studio Code: utilizado como ambiente principal para edição e organização dos arquivos do projeto.

Maven: utilizado no gerenciamento das dependências e na execução do projeto. O Maven Wrapper existente no próprio projeto também foi utilizado para executar os testes e iniciar a aplicação.

5. Desenvolvimento das Atividades

5.1 Organização inicial do projeto

A primeira etapa prática foi preparar e organizar o projeto para iniciar o desenvolvimento do AutoStock.

Durante essa etapa, procurei separar os arquivos de acordo com suas responsabilidades. O projeto passou a possuir pastas específicas para controllers, services, repositories, models e outros arquivos necessários para o funcionamento da aplicação.

No início, essa separação pareceu um pouco complexa, pois eu ainda estava aprendendo como cada parte se relacionava. Com o avanço do desenvolvimento, ficou mais fácil compreender que essa organização evita concentrar todo o código em um único arquivo e facilita a localização de possíveis problemas.

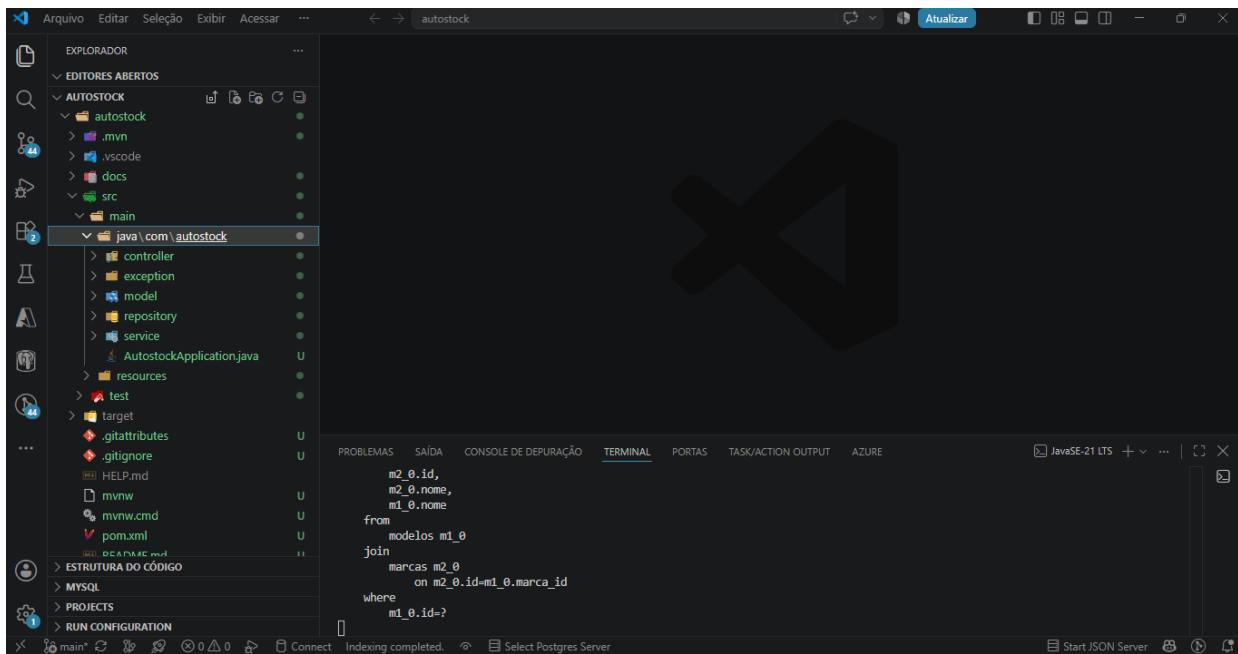


Figura 1 – Estrutura de pastas do projeto AutoStock no Visual Studio Code
Fonte: Elaborado pelo autor (2026).

A Figura 1 apresenta a estrutura principal do projeto AutoStock no Visual Studio Code. Nela é possível visualizar a organização das pastas e a separação das responsabilidades entre controller, model, repository e service. Essa organização ajudou a manter o projeto mais estruturado e facilitou a compreensão de como cada parte da aplicação participa do funcionamento do sistema.

5.2 Configuração do banco de dados

Para armazenar os dados de forma permanente, utilizei o MySQL.

Foi criado o banco de dados autostock_db, utilizado pela aplicação para armazenar as informações relacionadas às marcas, modelos e veículos.

A conexão entre o Spring Boot e o MySQL foi configurada no arquivo application.properties. Durante os testes também verifiquei diretamente pelo MySQL se o banco estava disponível e se os registros cadastrados pela aplicação estavam sendo armazenados.

Essa etapa me ajudou a compreender melhor a diferença entre as informações que aparecem na interface e os dados que realmente ficam registrados no banco.

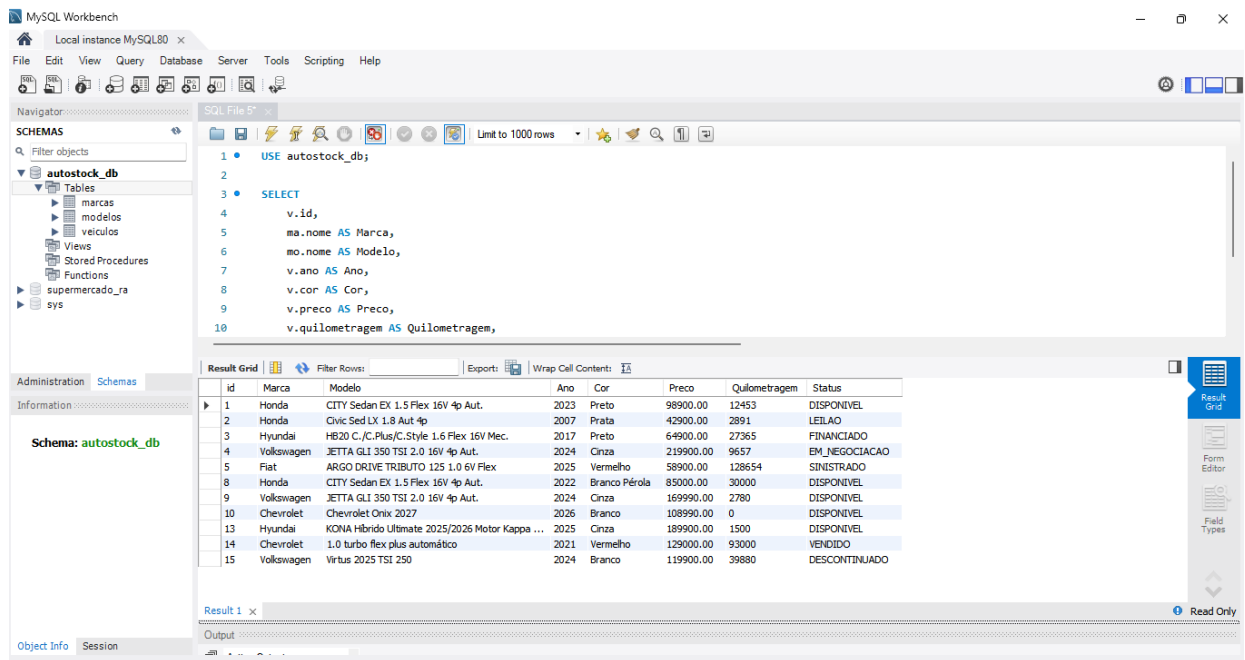


Figura 2 – Verificação do banco de dados AutoStock no MySQL
Fonte: Elaborado pelo autor (2026).

Agora vamos produzir a evidência do banco de dados que corresponde à seção 5.2 – Configuração do banco de dados. O relatório afirma que o autostock_db armazena marcas, modelos e veículos e que os registros foram verificados diretamente no MySQL.

5.3 Desenvolvimento da interface e execução do sistema

Depois da configuração das principais partes do projeto, a aplicação foi executada utilizando o Spring Boot.

O sistema ficou disponível localmente por meio do navegador. Na tela principal foram organizados os filtros, formulário de cadastro e a relação dos veículos existentes no estoque.

Ao chegar nessa etapa consegui visualizar de forma mais clara o resultado do desenvolvimento, pois as informações armazenadas no banco começaram a ser apresentadas na interface.

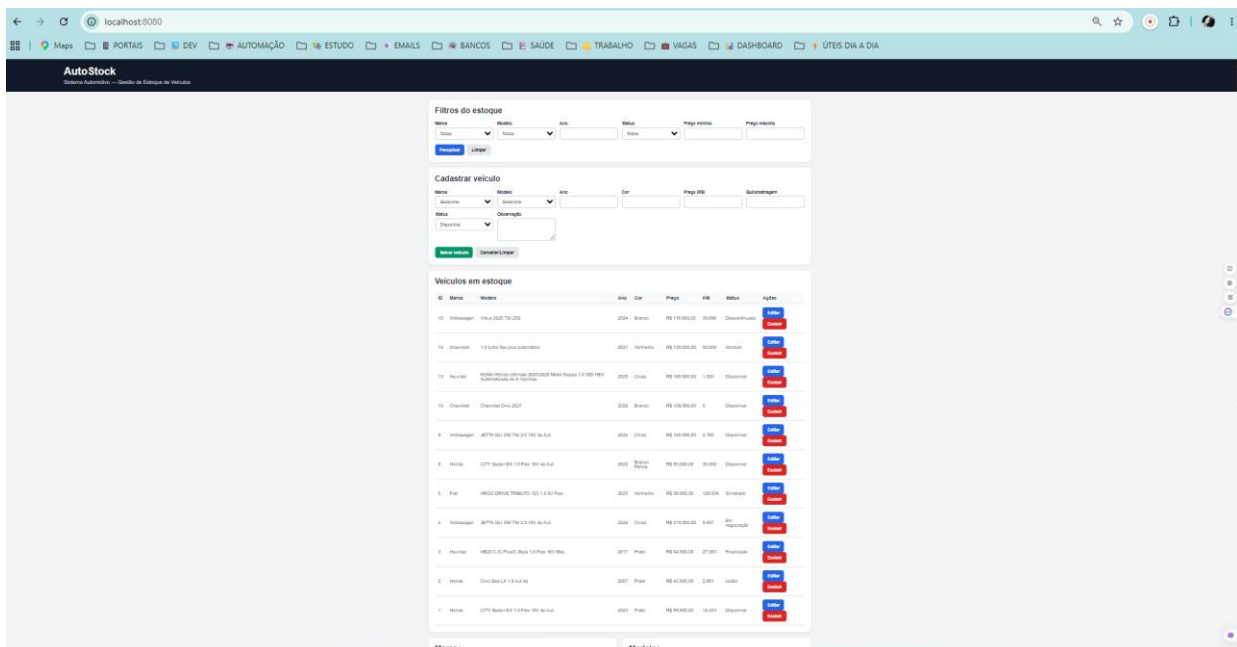


Figura 3 – Tela principal do sistema AutoStock em funcionamento
Fonte: Elaborado pelo autor (2026).

A Figura 3 apresenta a tela principal do AutoStock em funcionamento no navegador. Nela estão reunidos os filtros de pesquisa, o formulário para cadastro de veículos e a relação dos veículos existentes no estoque. A tela também disponibiliza as opções de edição e exclusão, permitindo realizar as principais operações do sistema em um único ambiente.

Após iniciar a aplicação, acessei o sistema AutoStock pelo navegador utilizando o endereço local. Nesta tela consegui visualizar os veículos cadastrados no estoque, além das opções de filtros, cadastro, edição e exclusão. Essa etapa foi importante para verificar de forma prática se as principais funções do sistema estavam funcionando e se as informações cadastradas no banco de dados estavam sendo apresentadas corretamente.

5.4 Cadastro de marcas e modelos

Uma das primeiras funcionalidades implementadas foi o cadastro de marcas e modelos.

O sistema permite cadastrar uma nova marca e posteriormente cadastrar os modelos correspondentes. Dessa maneira, o modelo fica relacionado à sua respectiva marca.

Durante os testes, por exemplo, cadastrei uma nova marca e posteriormente um modelo relacionado a ela. Após o cadastro, as informações ficaram disponíveis automaticamente nos campos utilizados para cadastrar os veículos.

Essa funcionalidade me ajudou a compreender melhor o relacionamento entre objetos e informações dentro do banco de dados.

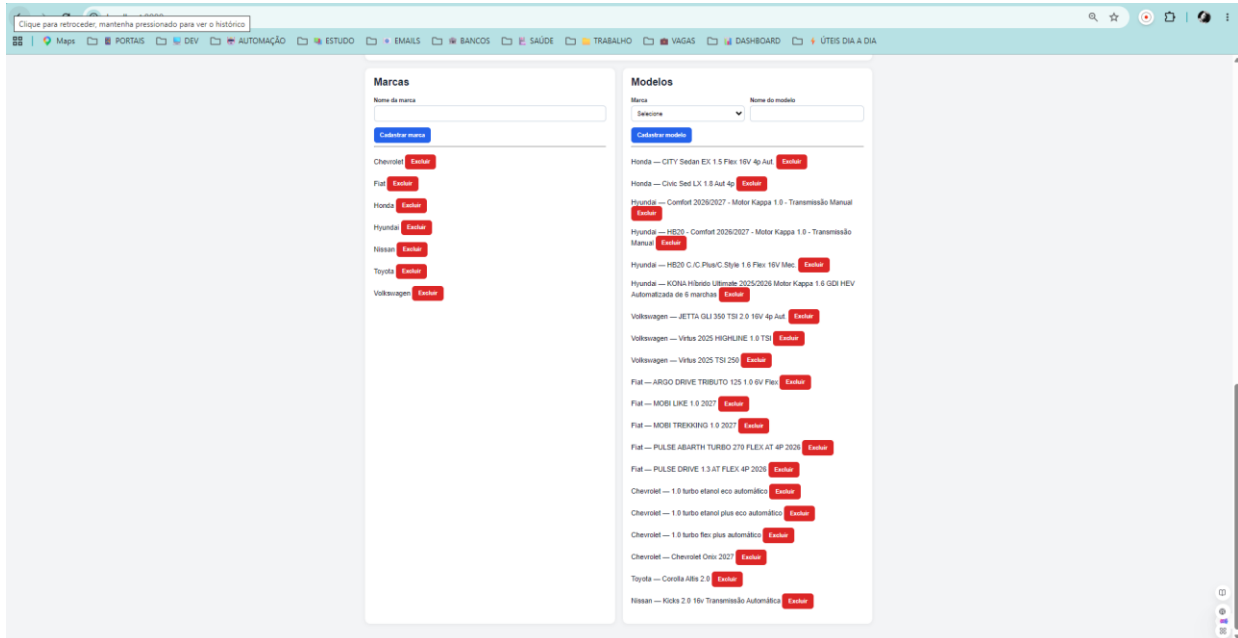


Figura 4 – Cadastro e relacionamento entre marcas e modelos
Fonte: Elaborado pelo autor (2026).

A Figura 4 apresenta o cadastro de marcas e modelos utilizado pelo AutoStock. Os modelos são associados às respectivas marcas, permitindo organizar essas informações antes do cadastro dos veículos. Esse relacionamento também faz com que, ao selecionar uma marca no formulário de veículos, sejam apresentados os modelos correspondentes.

5.5 Cadastro de veículos

O cadastro de veículos foi desenvolvido contendo os principais dados necessários para controle do estoque.

No formulário é possível informar marca, modelo, ano, cor, preço, quilometragem, status e observação.

Um ponto importante foi fazer com que o campo modelo dependesse da marca selecionada. Dessa maneira, ao escolher uma marca, o sistema apresenta os modelos correspondentes.

Depois de preencher as informações e salvar, o veículo passa a aparecer na relação de veículos em estoque e os dados também são armazenados no MySQL.

AutoStock
Sistema Automotivo — Gestão de Estoque de Veículos

Filtros do estoque

Marca: Todas | Modelo: Todos | Ano: | Status: Todos | Preço mínimo: | Preço máximo: |

Cadastrar veículo

Marca: Toyota | Modelo: Corolla Altis 2.0 | Ano: 2026 | Cor: Preto | Preço (R\$): 175.900,00 | Quilometragem: 850 | Status: Disponível | Observação: Veículo cadastrado para validação do sistema

Veículos em estoque

ID	Marca	Modelo	Ano	Cor	Preço	KM	Status	Ações
15	Volkswagen	Virtus 2025 TSI 250	2024	Branco	R\$ 119.900,00	39.880	Descontinuado	Editar Excluir
14	Chevrolet	1.0 turbo flex plus automático	2021	Vermelho	R\$ 129.000,00	93.000	Vendido	Editar Excluir

Figura 5 – Preenchimento dos dados para cadastro de um veículo
Fonte: Elaborado pelo autor (2026).

A Figura 5 apresenta o preenchimento do formulário utilizado para cadastrar um novo veículo no AutoStock. Foram informados os dados de marca, modelo, ano, cor, preço, quilometragem, status e observação. Também é possível observar que o modelo Corolla Altis 2.0 está relacionado à marca Toyota selecionada no formulário.

Veículos em estoque

ID	Marca	Modelo	Ano	Cor	Preço	KM	Status	Ações
16	Toyota	Corolla Altis 2.0	2026	Preto	R\$ 175.900,00	850	Disponível	Editar Excluir
15	Volkswagen	Virtus 2025 TSI 250	2024	Branco	R\$ 119.900,00	39.880	Descontinuado	Editar Excluir
14	Chevrolet	1.0 turbo flex plus automático	2021	Vermelho	R\$ 129.000,00	93.000	Vendido	Editar Excluir
13	Hyundai	KONA Híbrido Ultimate 2025/2026 Motor Kappa 1.6 GDI HEV Automatizada de 6 marchas	2025	Cinza	R\$ 189.900,00	1.500	Disponível	Editar Excluir
10	Chevrolet	Chevrolet Onix 2027	2026	Branco	R\$ 108.990,00	0	Disponível	Editar Excluir
9	Volkswagen	JETTA GLI 350 TSI 2.0 16V 4p Aut.	2024	Cinza	R\$ 169.990,00	2.780	Disponível	Editar Excluir

Figura 6 – Veículo cadastrado e apresentado no estoque
Fonte: Elaborado pelo autor (2026).

A Figura 6 apresenta o resultado do cadastro realizado anteriormente. Após salvar as informações, o veículo passou a fazer parte da relação de veículos em estoque, demonstrando que os dados informados no formulário foram processados e registrados corretamente pelo sistema.

5.6 Consulta e filtros do estoque

Outra funcionalidade desenvolvida foi a pesquisa de veículos utilizando filtros.

Foram disponibilizados filtros por marca, modelo, ano, status, preço mínimo e preço máximo.

Durante a validação realizei diferentes pesquisas para verificar o funcionamento. Ao selecionar uma marca, o sistema retornou somente os veículos daquela marca que realmente estavam cadastrados no estoque.

Também testei a combinação entre marca e modelo.

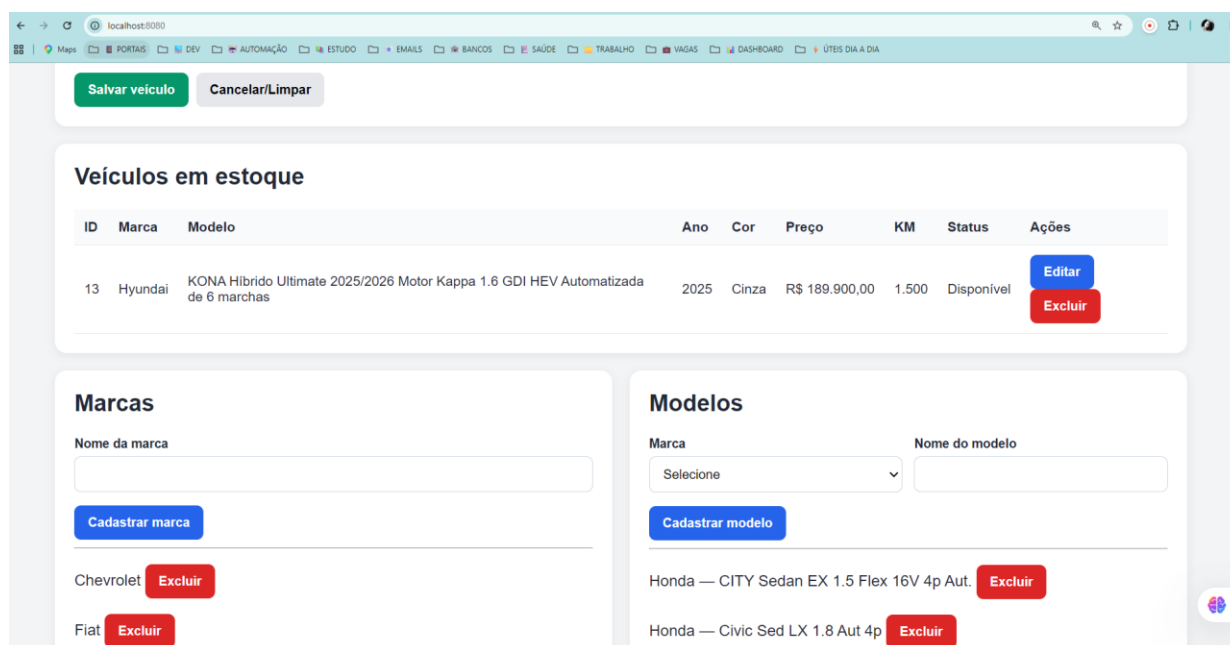


Figura 7 – Consulta de veículo utilizando os filtros de marca e modelo

Fonte: Elaborado pelo autor (2026).

A Figura 7 apresenta uma consulta realizada utilizando simultaneamente os filtros de marca e modelo. Ao selecionar a marca Hyundai e o modelo KONA, o sistema apresentou somente o veículo correspondente aos critérios informados. Esse teste demonstrou que os filtros podem ser combinados para tornar a pesquisa do estoque mais específica.

Também realizei uma pesquisa utilizando o ano do veículo. Ao pesquisar pelo ano de 2025, o sistema apresentou somente os veículos correspondentes ao período informado.

The screenshot shows a web application interface for managing a car stock. At the top, there's a navigation bar with various icons. The main content area is divided into several sections:

- Veículos em estoque:** A table listing cars with columns: ID, Marca, Modelo, Ano, Cor, Preço, KM, Status, and Ações. Two cars are listed, both from 2025, with 'Editar' and 'Excluir' buttons for each.
- Marcas:** A section for managing brands, including a text input for 'Nome da marca', a 'Cadastrar marca' button, and a list of existing brands (Chevrolet, Fiat) with 'Excluir' buttons.
- Modelos:** A section for managing models, including a dropdown for 'Marca', a text input for 'Nome do modelo', a 'Cadastrar modelo' button, and a list of existing models (Honda — CITY Sedan EX 1.5 Flex 16V 4p Aut., Honda — Civic Sed LX 1.8 Aut 4p) with 'Excluir' buttons.

Figura 8 – Consulta dos veículos utilizando filtro por ano
Fonte: Elaborado pelo autor (2026).

A Figura 8 apresenta a utilização do filtro por ano disponível no AutoStock. Ao informar o ano de 2025, o sistema retornou somente os veículos cadastrados com esse mesmo ano, demonstrando que a pesquisa permite localizar os veículos de acordo com o período informado.

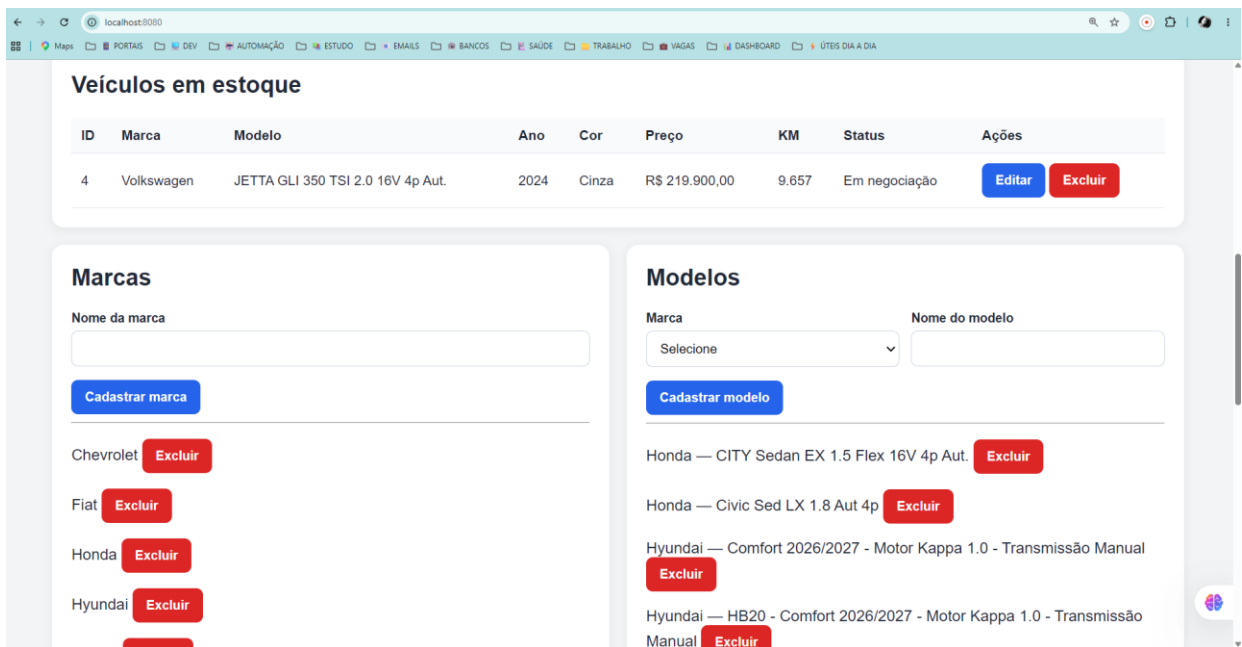


Figura 9 – Consulta dos veículos utilizando filtro por status
Fonte: Elaborado pelo autor (2026).

A Figura 9 apresenta a consulta realizada utilizando o filtro por status. Ao selecionar a opção “Em negociação”, o sistema exibiu somente os veículos cadastrados nessa situação. Esse recurso permite identificar de forma rápida os veículos de acordo com sua condição atual no estoque.

Por último, também foi testada a consulta por faixa de preço, informando um valor mínimo e um valor máximo. O resultado apresentou somente os veículos que estavam dentro da faixa pesquisada.

Veículos em estoque

ID	Marca	Modelo	Ano	Cor	Preço	KM	Status	Ações
16	Toyota	Corolla Altis 2.0	2026	Preto	R\$ 175.900,00	850	Disponível	Editar Excluir
15	Volkswagen	Virtus 2025 TSI 250	2024	Branco	R\$ 119.900,00	39.880	Descontinuado	Editar Excluir
14	Chevrolet	1.0 turbo flex plus automático	2021	Vermelho	R\$ 129.000,00	93.000	Vendido	Editar Excluir
13	Hyundai	KONA Híbrido Ultimate 2025/2026 Motor Kappa 1.6 GDI HEV Automatizada de 6 marchas	2025	Cinza	R\$ 189.900,00	1.500	Disponível	Editar Excluir
10	Chevrolet	Chevrolet Onix 2027	2026	Branco	R\$ 108.990,00	0	Disponível	Editar Excluir
9	Volkswagen	JETTA GLI 350 TSI 2.0 16V 4p Aut.	2024	Cinza	R\$ 169.990,00	2.780	Disponível	Editar Excluir

Figura 10 – Consulta dos veículos utilizando filtro por faixa de preço
 Fonte: Elaborado pelo autor (2026).

A Figura 10 apresenta uma consulta realizada por faixa de preço. Foram definidos os valores mínimo de R\$ 100.000,00 e máximo de R\$ 200.000,00, fazendo com que o sistema apresentasse somente os veículos que se encontravam dentro desse intervalo. Esse recurso permite realizar pesquisas no estoque de acordo com uma determinada faixa de valor.

5.7 Edição das informações

Além do cadastro e consulta, foi implementada a possibilidade de editar um veículo existente.

Ao clicar no botão Editar, os dados do veículo são carregados novamente no formulário. Dessa forma, é possível modificar as informações necessárias e salvar a atualização.

Durante os testes alterei informações como preço, quilometragem e status. Após salvar, o registro existente foi atualizado sem criar um novo veículo.

Editar veículo #16

Marca:
 Modelo:
 Ano:
 Cor:
 Preço (R\$):
 Quilometragem:

Status:
 Observação:

Veículos em estoque

ID	Marca	Modelo	Ano	Cor	Preço	KM	Status	Ações
16	Toyota	Corolla Altis 2.0	2026	Preto	R\$ 175.900,00	850	Disponível	Editar Excluir
15	Volkswagen	Virtus 2025 TSI 250	2024	Branco	R\$ 119.900,00	39.880	Descontinuado	Editar Excluir

Figura 11 – Alteração das informações de um veículo cadastrado
 Fonte: Elaborado pelo autor (2026).

A Figura 11 apresenta a utilização da função de edição do AutoStock. Ao selecionar a opção “Editar” em um veículo cadastrado, suas informações são carregadas novamente no formulário, permitindo realizar as alterações necessárias. Após a atualização, o sistema modifica o registro existente sem realizar um novo cadastro.

5.8 Exclusão de veículos

O sistema também permite excluir veículos cadastrados.

Para validar essa funcionalidade, utilizei um registro criado durante os testes. Após selecionar a opção Excluir, o veículo deixou de aparecer no estoque.

Também observei que a exclusão do veículo não removeu automaticamente a marca e o modelo cadastrados, pois são informações independentes que podem ser utilizadas posteriormente para outros veículos.

ID	Marcas	Modelos	Ano	Cor	Preço	KM	Status	Ações
15	Volkswagen	Virtus 2025 TSI 250	2024	Branco	R\$ 119.900,00	39.880	Descontinuado	Editar Excluir
14	Chevrolet	1.0 turbo flex plus automática	2021	Vermelho	R\$ 129.000,00	93.000	Vendido	Editar Excluir
13	Hyundai	KONA Híbrido Ultimate 2025/2026 Motor Kappa 1.6 GDI HEV Automatizada de 8 marchas	2025	Cinza	R\$ 189.900,00	1.500	Disponível	Editar Excluir
10	Chevrolet	Chevrolet Onix 2027	2026	Branco	R\$ 105.990,00	0	Disponível	Editar Excluir
9	Volkswagen	JETTA GLI 350 TSI 2.0 16V 4p Aut.	2024	Cinza	R\$ 169.990,00	2.780	Disponível	Editar Excluir
8	Honda	CITY Sedan EX 1.5 Flex 16V 4p Aut.	2022	Branco Pérola	R\$ 85.990,00	30.000	Disponível	Editar Excluir
5	Fiat	ARGO DRIVE TRIBUTO 125 1.0 6V Flex	2025	Vermelho	R\$ 58.990,00	126.654	Sinistrado	Editar Excluir
4	Volkswagen	JETTA GLI 350 TSI 2.0 16V 4p Aut.	2024	Cinza	R\$ 219.900,00	9.657	Em negociação	Editar Excluir
3	Hyundai	HB20 C/C Plus/C-Style 1.6 Flex 16V Mec.	2017	Preto	R\$ 64.900,00	27.365	Financiado	Editar Excluir
2	Honda	Civic Sed LX 1.8 Aut 4p	2007	Prata	R\$ 42.900,00	2.891	Leilão	Editar Excluir
1	Honda	CITY Sedan EX 1.5 Flex 16V 4p Aut.	2023	Preto	R\$ 95.990,00	12.453	Disponível	Editar Excluir

Marcas: Modelos:

Figura 12 – Opção utilizada para exclusão de um veículo do estoque
Fonte: Elaborado pelo autor (2026).

A Figura 12 apresenta a opção utilizada para excluir um veículo cadastrado no AutoStock. Para realizar esse teste, foi utilizado o veículo Toyota Corolla Altis 2.0 cadastrado anteriormente durante a validação do sistema. Após selecionar a opção “Excluir”, o registro foi removido da relação de veículos em estoque, demonstrando o funcionamento da operação de exclusão.

6. Aplicação dos Conceitos de Programação Orientada a Objetos

Durante o desenvolvimento procurei aplicar os conceitos de Programação Orientada a Objetos estudados na disciplina.

6.1 Classes e objetos

Foram utilizadas classes para representar as principais informações do sistema, como Marca, Modelo e Veículo.

Cada classe representa um tipo de informação do AutoStock. Quando um veículo é cadastrado, por exemplo, os dados informados passam a representar um objeto com suas próprias características.

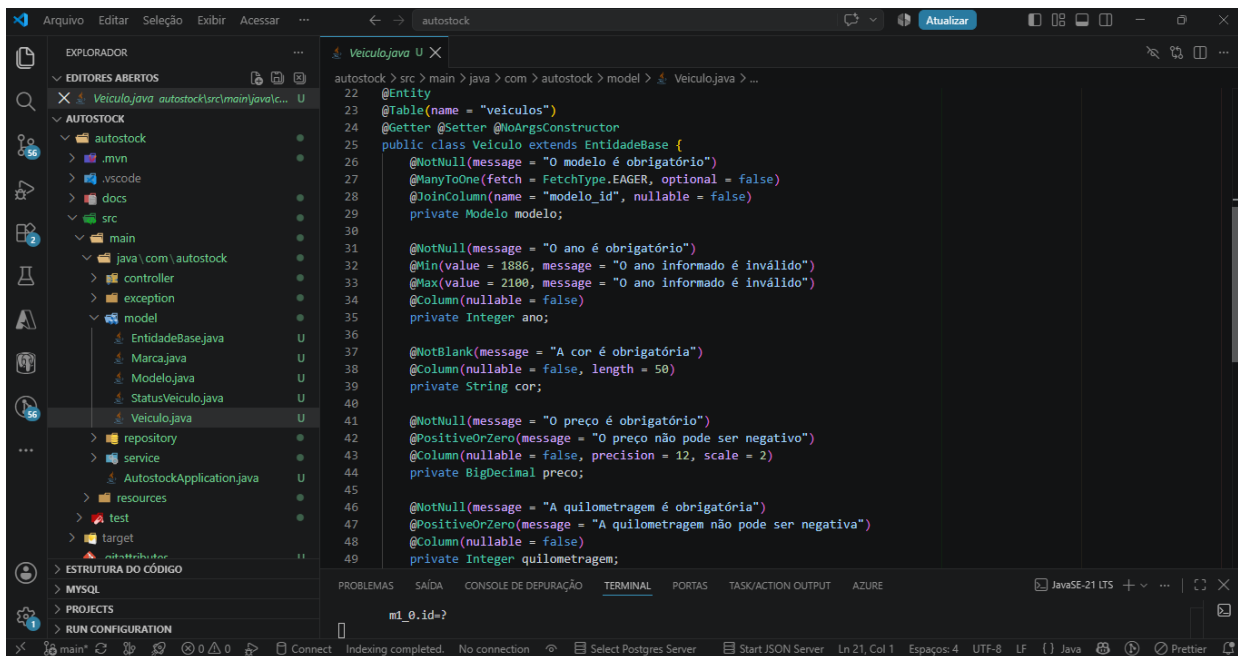


Figura 13 – Classe utilizada para representar os veículos no sistema
Fonte: Elaborado pelo autor (2026).

A Figura 13 apresenta a classe Veiculo, utilizada para representar os veículos cadastrados no AutoStock. Nela foram definidos atributos como modelo, ano, cor, preço, quilometragem, status e observação. Esses atributos representam as principais características necessárias para armazenar e manipular as informações dos veículos dentro do sistema. A classe também possui o relacionamento com a classe Modelo, permitindo associar cada veículo ao modelo correspondente.

6.2 Atributos e métodos

Os atributos representam as características armazenadas nos objetos.

No caso do veículo, são utilizadas informações como ano, cor, preço, quilometragem, status e observação, além do relacionamento com o modelo.

Os métodos são utilizados para permitir que essas informações sejam acessadas ou modificadas e também para executar determinadas operações dentro da aplicação.

6.3 Encapsulamento

O encapsulamento foi aplicado mantendo os atributos das classes protegidos do acesso direto e utilizando métodos para acessar e alterar essas informações.

Durante o desenvolvimento, esse conceito me ajudou a compreender que os dados de um objeto não precisam ficar totalmente expostos para todas as partes da aplicação.

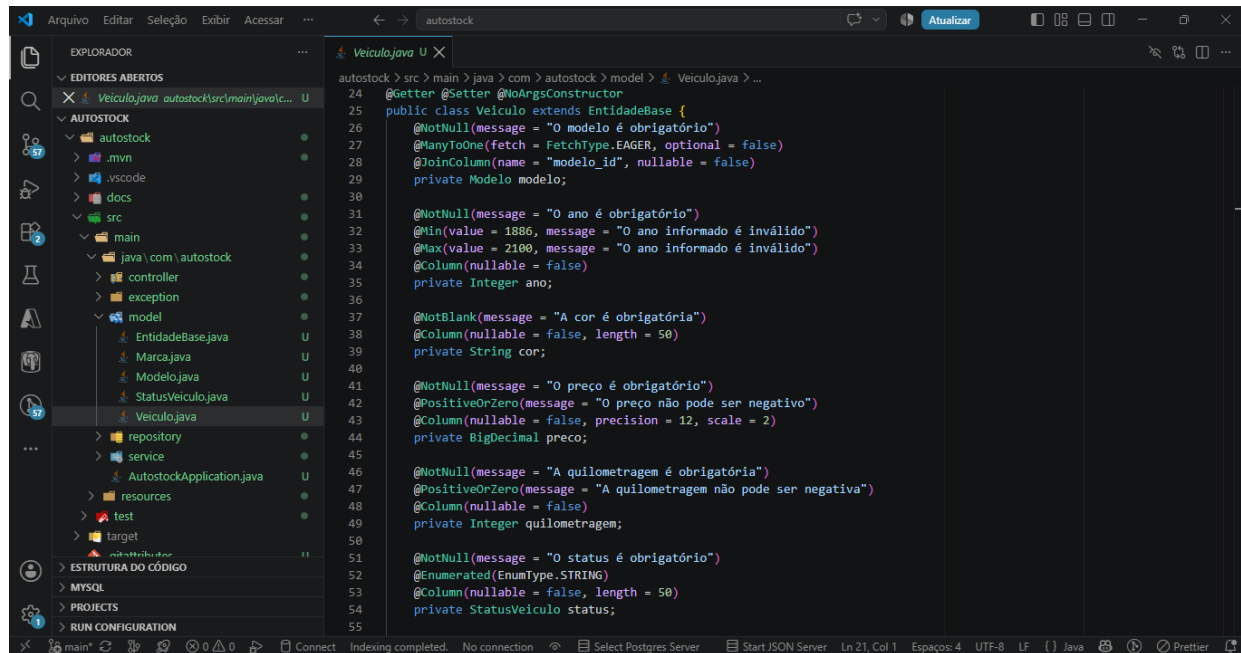


Figura 14 – Exemplo da aplicação de encapsulamento no projeto
Fonte: Elaborado pelo autor (2026).

A Figura 14 apresenta a aplicação do encapsulamento na classe Veiculo. Os atributos foram definidos como private, evitando o acesso direto aos dados da classe. Para permitir o acesso e a alteração dessas informações, foram utilizadas as anotações @Getter e @Setter da biblioteca Lombok, que geram automaticamente os métodos de leitura e alteração dos atributos. Dessa forma, foi possível aplicar o encapsulamento sem a necessidade de escrever manualmente todos os métodos getters e setters.

6.4 Herança

A herança foi utilizada na estrutura orientada a objetos do projeto por meio da classe base definida para compartilhar características comuns entre entidades do sistema.

Esse recurso permite que uma classe possa aproveitar características existentes em outra classe, evitando repetição desnecessária de código.

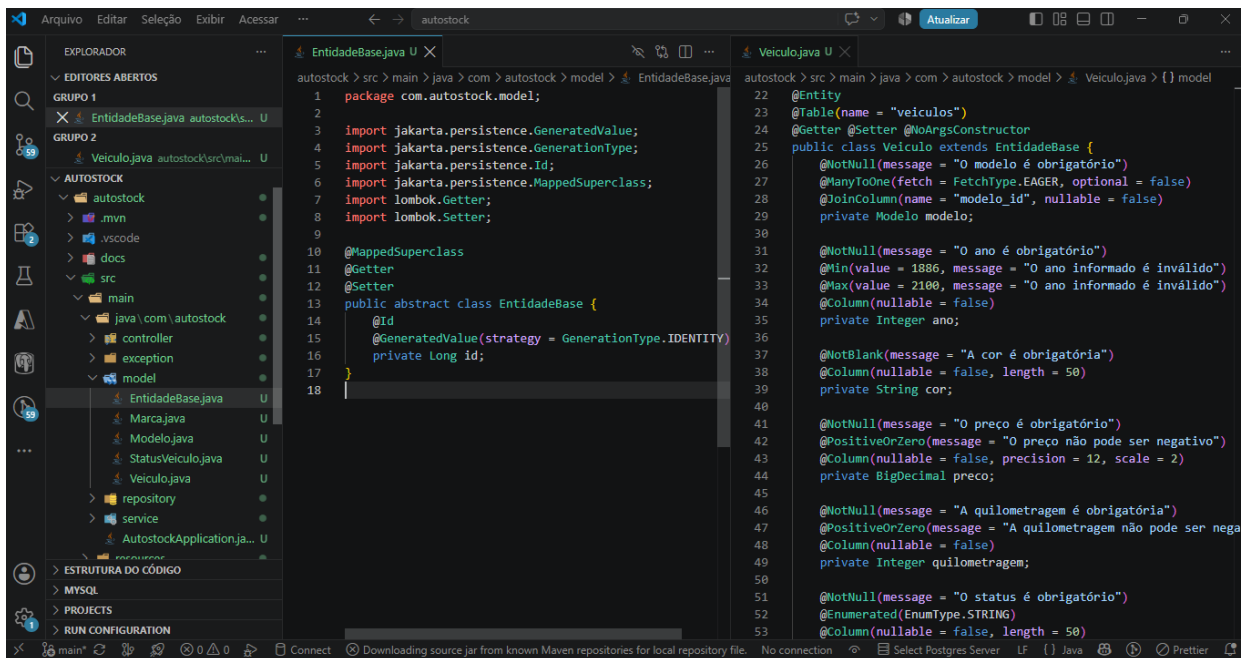


Figura 15 – Aplicação do conceito de herança no AutoStock
Fonte: Elaborado pelo autor (2026).

A Figura 15 apresenta a aplicação do conceito de herança no AutoStock. Foi criada a classe abstrata *EntidadeBase*, responsável por definir o atributo *id*, utilizado para identificar os registros no banco de dados. A classe *Veiculo* utiliza *extends EntidadeBase*, herdando essa característica. Dessa forma, foi possível reaproveitar uma característica comum entre as entidades e evitar a repetição do mesmo atributo em diferentes classes.

6.5 Relacionamento entre classes

Também foi necessário trabalhar com relacionamentos entre as classes.

Uma marca pode possuir diferentes modelos, enquanto cada modelo pertence a uma determinada marca. O veículo, por sua vez, utiliza um modelo cadastrado.

Essa estrutura evita a necessidade de repetir todas as informações da marca dentro de cada veículo e mantém os dados mais organizados.

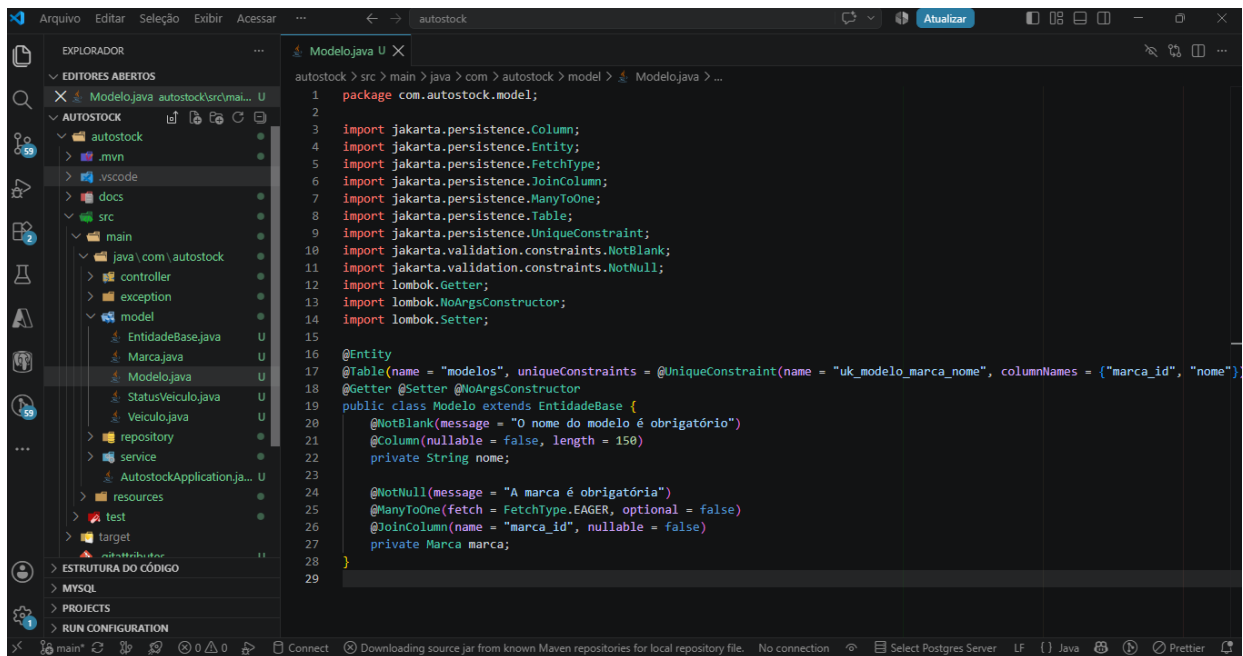


Figura 16 – Relacionamento entre as entidades Marca e Modelo
Fonte: Elaborado pelo autor (2026).

A Figura 16 apresenta o relacionamento entre as classes Modelo e Marca. Na classe Modelo, o atributo marca estabelece a associação com a classe Marca. A anotação @ManyToOne indica que vários modelos podem estar relacionados a uma mesma marca, enquanto @JoinColumn define a coluna marca_id utilizada para realizar esse relacionamento no banco de dados. Dessa forma, cada modelo cadastrado no AutoStock fica associado à sua respectiva marca.

7. Organização das Camadas da Aplicação

Durante o desenvolvimento também procurei manter diferentes responsabilidades separadas dentro do projeto.

O Controller recebe as requisições realizadas pela interface.

O Service concentra parte das regras e operações utilizadas pelo sistema.

O Repository é utilizado para realizar a comunicação necessária com o banco de dados.

As classes do Model representam as principais entidades e informações do AutoStock.

No começo tive dificuldade para entender por que era necessário separar essas partes. Com os testes realizados durante o desenvolvimento, comecei a compreender que cada camada possui uma responsabilidade e que elas precisam trabalhar juntas.

O fluxo pode ser entendido de forma simplificada como:

Interface → Controller → Service → Repository → Banco de Dados

e, posteriormente, as informações retornam para serem apresentadas novamente ao usuário.

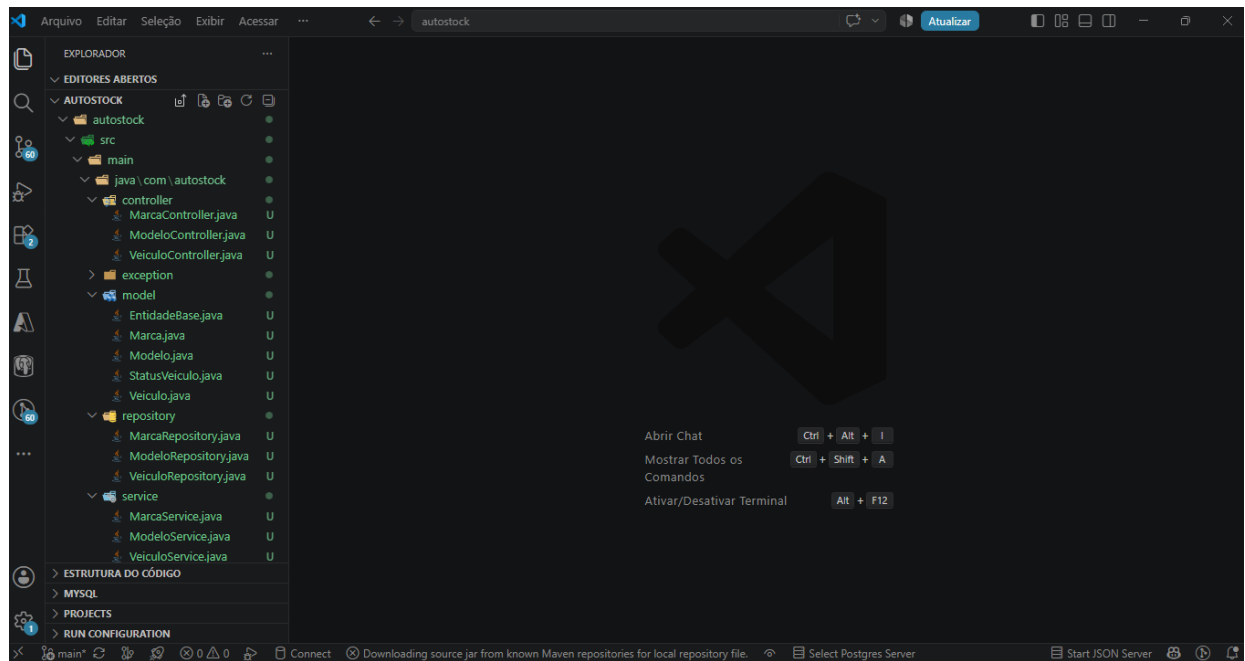


Figura 17 – Organização das principais camadas do AutoStock

Fonte: Elaborado pelo autor (2026).

A Figura 17 apresenta a organização das principais camadas utilizadas no desenvolvimento do AutoStock. A camada controller é responsável pelo recebimento das requisições, a camada service concentra as regras de negócio, a camada repository realiza o acesso aos dados e a camada model representa as entidades utilizadas pelo sistema. Essa separação contribuiu para manter o código organizado e facilitar a compreensão da estrutura da aplicação.

8. Testes e Validação

Após concluir as principais funcionalidades, realizei testes para verificar se as diferentes partes do projeto estavam funcionando juntas.

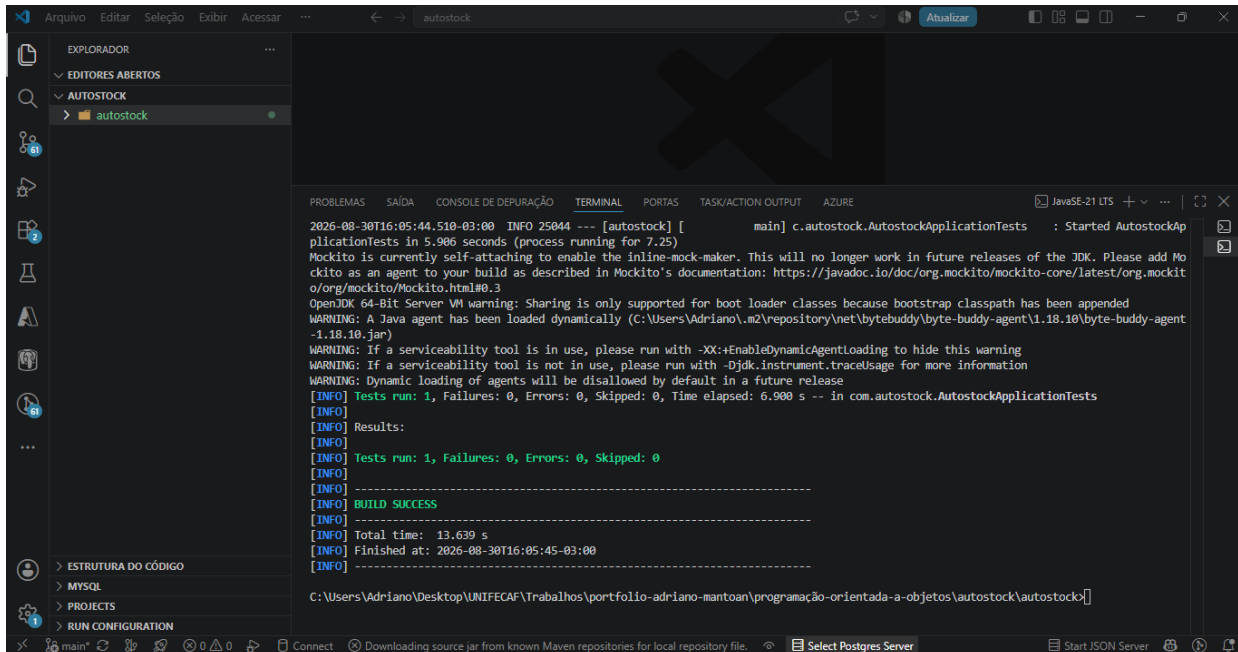
Foram testados cadastro de marcas, cadastro de modelos, cadastro de veículos, edição, exclusão e todos os filtros disponíveis.

Também utilizei o Maven Wrapper para realizar um teste da aplicação.

O comando utilizado foi:

.\mvnw.cmd clean test

Ao final da execução, o Maven apresentou BUILD SUCCESS. O teste executado terminou com 1 teste realizado, nenhuma falha e nenhum erro, confirmando que o contexto da aplicação foi carregado corretamente.



```
2026-08-30T16:05:44.518-03:00 INFO 25944 --- [autostock] [main] c.autostock.AutostockApplicationTests : Started AutostockAp
plicationTests in 5.906 seconds (process running for 7.25)
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mo
ckito as an agent to your build as described in Mockito's documentation: https://javadoc.io/doc/org.mockito/mockito-core/latest/org.mockit
o/org.mockito/Mockito.html#0.3
OpenJDK 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
WARNING: A Java agent has been loaded dynamically (C:\Users\Adriano\.m2\repository\net\bytebuddy\byte-buddy-agent\1.18.10\byte-buddy-agent
-1.18.10.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 6.900 s -- in com.autostock.AutostockApplicationTests
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO]
[INFO] Total time: 13.639 s
[INFO] Finished at: 2026-08-30T16:05:45-03:00
[INFO]
[INFO] -----
C:\Users\Adriano\Desktop\UNIFECAF\Trabalhos\portfolio-adriano-mantoan\programação-orientada-a-objetos\autostock\autostock>]
```

Figura 18 – Resultado da execução dos testes do projeto pelo Maven
Fonte: Elaborado pelo autor (2026).

A Figura 18 apresenta o resultado dos testes executados no projeto por meio do Maven Wrapper. Durante a execução foi realizado um teste da aplicação, sem ocorrência de falhas ou erros. A mensagem BUILD SUCCESS confirmou que o projeto foi compilado e testado corretamente nesse processo de validação.

Também foi possível verificar durante os testes que a aplicação estava conseguindo estabelecer conexão com o banco autostock_db utilizando o MySQL.

9. Dificuldades Encontradas

Durante o desenvolvimento do AutoStock encontrei algumas dificuldades, principalmente por ainda estar adquirindo experiência com Java, Spring Boot, banco de dados e organização de aplicações em diferentes camadas.

Uma das primeiras dificuldades foi compreender como as classes deveriam se relacionar e como separar as responsabilidades entre Controller, Service, Repository e Model.

Também tive dificuldades relacionadas à configuração do ambiente de desenvolvimento. Durante o projeto foi necessário verificar a versão correta do Java e entender como executar o Maven Wrapper disponível dentro da própria pasta do projeto.

Outro ponto que exigiu atenção foi a comunicação com o MySQL. Foi necessário conferir se o banco estava disponível, se a aplicação conseguia estabelecer a conexão e se os dados cadastrados realmente estavam sendo armazenados.

Durante a validação dos filtros também tive uma dúvida ao pesquisar uma determinada marca. Inicialmente imaginei que o sistema não estava mostrando todos os veículos. Depois de consultar diretamente o banco de dados, percebi que existiam vários modelos cadastrados para aquela marca, mas somente um veículo daquela marca estava efetivamente cadastrado no estoque. Essa verificação me ajudou a entender melhor a diferença entre o cadastro de modelos e o cadastro dos veículos.

Também ocorreu uma situação em que tentei executar um comando do Maven enquanto ainda estava dentro do terminal do MySQL. Depois identifiquei que comandos como `.mvnw.cmd` precisam ser executados pelo PowerShell e não pelo MySQL.

Em outro momento, ao tentar iniciar novamente a aplicação, foi informado que a porta 8080 já estava sendo utilizada. Após verificar a situação, identifiquei que já existia uma instância do AutoStock funcionando naquela porta. Isso mostrou a importância de observar as mensagens apresentadas pelo terminal antes de realizar alterações no código. O registro dessa ocorrência mostra explicitamente que a segunda inicialização falhou porque a porta 8080 já estava ocupada.

Essas dificuldades fizeram parte do aprendizado e contribuíram para que eu pudesse compreender melhor o funcionamento do projeto e a comunicação entre suas diferentes partes.

10. Análise dos Resultados

Após os testes realizados, foi possível verificar que o AutoStock conseguiu atender às principais funcionalidades definidas para o projeto.

O sistema permite cadastrar marcas e modelos, relacionar os modelos às respectivas marcas e utilizar essas informações posteriormente no cadastro dos veículos.

Também foi possível cadastrar veículos contendo suas principais informações e visualizar esses registros diretamente na tela de estoque.

Os filtros funcionaram de acordo com os testes realizados, permitindo pesquisar por marca, modelo, ano, status e faixa de preço.

A edição permitiu modificar as informações de um registro existente sem criar um novo veículo, enquanto a exclusão permitiu remover um veículo do estoque.

Outro resultado importante foi conseguir fazer com que a interface, as classes Java e o banco MySQL trabalhassem em conjunto. Para mim, essa foi uma das partes mais importantes do projeto, pois permitiu visualizar na prática conteúdos que anteriormente eram compreendidos principalmente de forma teórica.

11. Conclusão

O desenvolvimento do AutoStock foi importante para colocar em prática os conhecimentos apresentados durante a disciplina de Programação Orientada a Objetos.

Durante o trabalho consegui compreender melhor a utilização de classes, objetos, atributos, métodos, encapsulamento, relacionamentos entre entidades e a separação das responsabilidades dentro de uma aplicação.

Também pude ter um contato maior com Java, Spring Boot, Maven e MySQL, além de compreender melhor como uma interface pode se comunicar com o backend e como as informações podem ser armazenadas em um banco de dados.

Mesmo encontrando dificuldades durante o desenvolvimento, os problemas identificados foram sendo analisados e corrigidos gradualmente. Esse processo também fez parte do aprendizado, pois muitas vezes foi necessário interpretar mensagens apresentadas no terminal e verificar diferentes partes do projeto até encontrar a origem de um problema.

Ao final, o sistema apresentou funcionamento para cadastro, consulta, edição, exclusão e filtros dos veículos, atendendo às principais funcionalidades propostas para o AutoStock.

Considero que o projeto contribuiu para minha evolução na disciplina, principalmente por permitir sair somente da parte teórica e acompanhar o funcionamento de uma aplicação completa, desde o cadastro realizado pelo usuário até o armazenamento e consulta das informações no banco de dados.

12. Referências

ORACLE. **Java Platform, Standard Edition 21 Documentation**. Disponível em: [Java SE 21 Documentation](#). Acesso em: 30 ago. 2026.

SPRING. **Spring Boot Reference Documentation**. Disponível em: [Spring Boot Reference Documentation](#). Acesso em: 30 ago. 2026.

SPRING. **Spring Data JPA – Reference Documentation**. Disponível em: [Spring Data JPA Reference Documentation](#). Acesso em: 30 ago. 2026.

ORACLE. **MySQL 8.0 Reference Manual**. Disponível em: [MySQL 8.0 Reference Manual](#). Acesso em: 30 ago. 2026.

APACHE SOFTWARE FOUNDATION. **Apache Maven Documentation**. Disponível em: [Apache Maven](#). Acesso em: 30 ago. 2026.

PROJECT LOMBOK. **Project Lombok Documentation**. Disponível em: [Project Lombok](#). Acesso em: 30 ago. 2026.

E deixe provisoriamente a última referência assim no Word:

MANTOAN, Adriano. **AutoStock – Sistema Automotivo de Gestão de Estoque de Veículos**. GitHub, 2026. Disponível em: https://github.com/adrianomantoan-tst-eng-comp2026/portfolio-adriano-mantoan/tree/main/programa%C3%A7%C3%A3o-orientada-a-objetos/autostock/autostock?utm_source=chatgpt.com. Acesso em: 30 ago. 2026.